

Hate Speech and Toxicity Detection Using DistilBERT:

A Multi-Label Classification System

Course Project Proposal – Phase 1

Jerin Pious, Evan Tomy

Computer Programming, Conestoga College

PROG74040 – Advanced Topics in Artificial Intelligence and Machine Learning

Akrem El-ghazal

June 23, 2026

1. Problem Definition

Online platforms process hundreds of millions of user-generated posts, comments, and messages each day, and a meaningful share of that content is hateful, abusive, or toxic. Manual review teams cannot keep pace: response times lag and harmful content stays live long enough to cause real damage.

Existing automated tools fall short in different ways. Keyword blocklists catch obvious slurs but miss the coded, context-dependent hostility that makes up most real-world hate speech. A comment like "people like that shouldn't be allowed here" contains no flagged words, yet its intent is unmistakably hostile in the right context. Rule-based systems cannot reason about that. Classical machine learning approaches, such as TF-IDF with logistic regression and naive Bayes, improve on keyword matching but still treat language as a bag of tokens rather than a sequence of meaning.

This project builds a multi-label toxicity classifier that identifies six distinct categories of harmful content in a single pass: toxic, severe toxic, obscene, threatening, insulting, and identity-based hate. The system is designed for deployment as a live REST API so that any application like a social platform, a moderation dashboard, or an educational tool can call it in real time. Pre-trained transformer models like DistilBERT (Sanh et al., 2019) have already internalized the statistical structure of natural language across billions of sentences. Fine-tuning such a model on labeled toxicity data allows it to recognize that "go back to where you came from" is hostile even though no individual word appears on any blocklist. That level of contextual reasoning is not available to shallower approaches.

Business and Social Impact

The consequences of poor content moderation are measurable. Platforms that fail to moderate toxic content face regulatory pressure, advertiser withdrawal, and user abandonment. The European Union's Digital Services Act (European Parliament, 2022) and Canada's proposed Online Harms Act (Government of Canada, 2024) both impose obligations on large platforms to proactively address hate speech at scale. A reliable, explainable classifier directly supports that compliance work.

Beyond regulatory compliance, there is a clear operational case. A classifier that pre-screens content before it goes live or flags it instantly for human review lets a moderation team focus on genuinely ambiguous cases rather than spending time on clear violations. That is a more efficient use of human judgment, and it scales in a way that purely human moderation cannot.

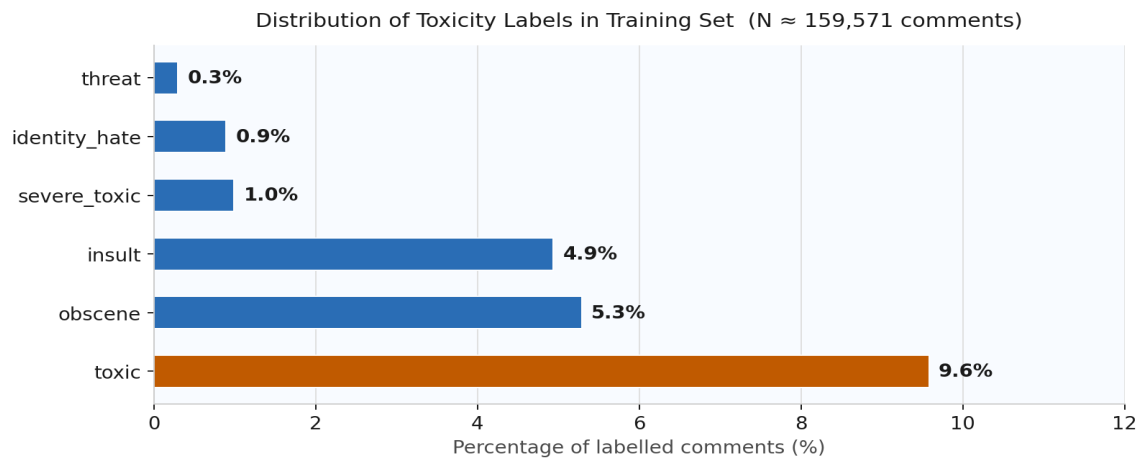
2. Dataset Description

Dataset

The dataset for this project is the Jigsaw Toxic Comment Classification Challenge, released by Google Jigsaw in 2018 (Jigsaw/Conversation AI, 2018). It contains approximately 160,000 comments drawn from Wikipedia talk pages, each annotated by human raters across six binary toxicity labels. The dataset is publicly available under a CC0 license (Jigsaw/Conversation AI, 2018), placing no restrictions on use, modification, or redistribution. Table 1 summarizes its key properties.

Table 1*Properties of the Jigsaw Toxic Comment Classification Dataset*

Property	Details
Total samples	~160,000 Wikipedia talk page comments
Label structure	Six binary labels per comment (multi-label output)
Labels	toxic, severe_toxic, obscene, threat, insult, identity_hate
Class distribution	~90% non-toxic; remaining 10% distributed across six categories
Language	English
License	CC0 — fully open for research and commercial use

Figure 1*Class Distribution of Toxicity Labels in the Jigsaw Training Set*

Note. N ≈ 159,571 comments. Labels are not mutually exclusive; a single comment may belong to multiple categories. The non-toxic majority (≈90%) is not shown. Source: Jigsaw/Conversation AI (2018).

The class distribution is the most significant data challenge in this project. With roughly 90% of comments labeled non-toxic, a naive model can achieve high accuracy by predicting the majority class on every input. This project addresses the imbalance through class-weighted loss and stratified sampling during training.

Preprocessing Pipeline

Table 2 outlines each step of the preprocessing pipeline that transforms raw comment text into the format required by the DistilBERT tokenizer.

Table 2

Preprocessing Pipeline Steps, Methods, and Rationale

Step	Method	Rationale
Text cleaning	Remove HTML tags, normalize Unicode	Raw comments contain wiki markup and encoding artifacts
Tokenization	Hugging Face DistilBERT tokenizer	Converts strings to input IDs and attention masks
Sequence padding	Truncate / pad to 256 tokens	Sentence-level input rarely exceeds this; saves memory
Label encoding	Multi-hot binary arrays (length 6)	Required for multi-label sigmoid output layer
Imbalance handling	Class weights + stratified splits	Prevents the model from collapsing to the majority class
Data augmentation	Back-translation (EN→FR→EN) + LLM-paraphrased synthetic samples for minority classes	Expands minority-class examples without manual labeling; improves recall on rare categories

3. Machine Learning Approach

3.1 Functional Requirements

The system must satisfy the following functional requirements:

- Accept raw text input—a comment, post, or sentence—via a REST API endpoint
- Classify the input across six toxicity categories in a single forward pass
- Return per-class confidence scores (0.0–1.0) as a JSON payload
- Support batch inference for bulk moderation workflows

3.2 Non-Functional Requirements

Table 3 defines the non-functional requirements governing inference performance, resource usage, and cost.

Table 3

Non-Functional Requirements and Targets

Requirement	Target
Inference Latency	< 500ms per warm request on CPU
Model Memory Footprint	< 300MB (DistilBERT on disk)
Availability	Single EC2 instance, best-effort uptime
Cost	AWS Free Tier only — zero ongoing spend

3.3 Model Architecture

The primary model is DistilBERT fine-tuned for multi-label classification. DistilBERT is a distilled version of BERT that retains 97% of BERT's language understanding capability while being 40% smaller and 60% faster in inference (Sanh et al., 2019, p. 1). That trade-off is what makes free-tier deployment feasible: a full BERT or RoBERTa model would exceed the memory constraints of an AWS t2.micro instance, while DistilBERT fits comfortably within them.

The multi-label output requires sigmoid activation rather than softmax. Softmax forces class probabilities to sum to 1.0, which implies exactly one correct label per input. Because a comment can simultaneously be obscene and threatening, sigmoid computes an independent probability per class—the correct formulation for multi-label output. The model is trained with BCEWithLogitsLoss, which applies a numerically stable sigmoid internally and computes independent classification loss per label.

3.4 Baseline Models

Two baseline models will be trained before the transformer. This establishes a concrete performance floor to measure improvement against and justifies the added complexity of fine-tuning a language model.

- TF-IDF + Logistic Regression, the simplest possible machine learning baseline; fast to train and easy to interpret
- LSTM with GloVe 100-dimensional embeddings (Pennington et al., 2014) a sequential model that understands word order but requires no GPU

3.5 Training Configuration

Table 4 summarizes the hyperparameters and the rationale for each choice.

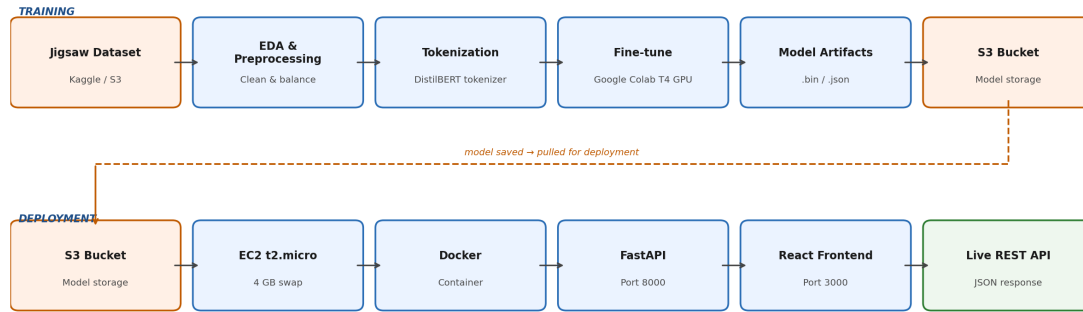
Table 4

Training Hyperparameters and Rationale

Hyperparameter	Value	Reason
Base model	distilbert-base-uncased	Speed/accuracy trade-off suited to CPU deployment
Max sequence length	256 tokens	Sentence-level input rarely exceeds this; memory-efficient
Dropout rate	0.3	Prevents overfitting on limited dataset size
Loss function	BCEWithLogitsLoss	Multi-label independent classification; numerically stable
Optimizer	AdamW	Standard for transformer fine-tuning
Learning rate	2e-5	Established best practice for BERT-family models
Epochs	3–5	Sufficient for fine-tuning; avoids catastrophic forgetting
Batch size	16	Memory-safe for Google Colab T4 training

Figure 3*Machine Learning Pipeline: Training and Deployment Flow*

ML Pipeline: Training and Deployment Flow



Note. The Amazon S3 bucket serves as the handoff point between the offline training pipeline (Google Colab T4 GPU) and the online deployment pipeline (AWS EC2 t2.micro). Dashed orange line indicates the model artifact transfer path.

3.6 Advanced Techniques

Three advanced techniques are planned beyond basic fine-tuning:

- SHAP / Integrated Gradients (Lundberg & Lee, 2017) token-level attribution to explain which words drove a prediction, making the model auditable
- Sigmoid threshold tuning, swept from 0.3 to 0.6 per class to optimize F1 on the validation set
- Multi-task learning a shared encoder with six task-specific heads will be explored as an alternative to a single multi-label head

3.7 Evaluation Metrics

Table 5 lists the evaluation metrics, the reason each was chosen, the target threshold, and its role in the evaluation strategy.

Table 5*Evaluation Metrics, Targets, and Roles*

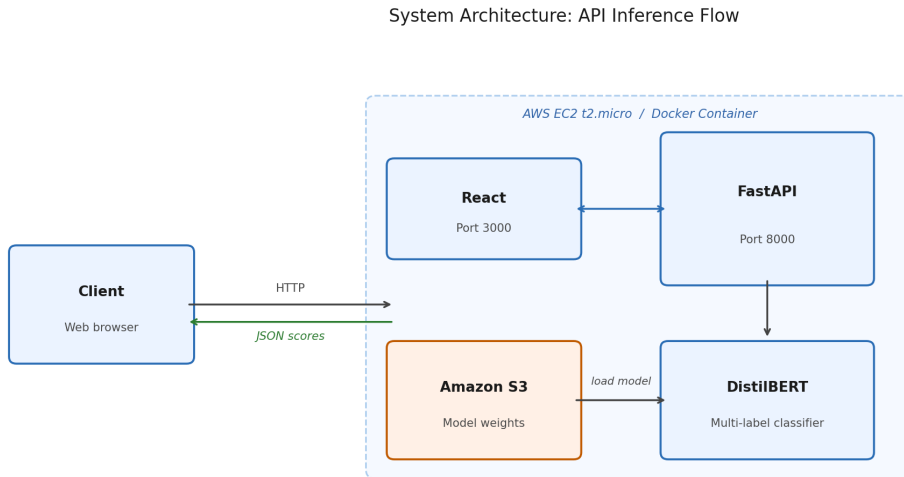
Metric	Why It Matters	Target	Role
ROC-AUC (macro)	Handles class imbalance; standard Jigsaw benchmark metric	> 0.95	Primary
F1-Score (macro)	Balances precision and recall across all six labels	> 0.82	Primary
Precision / Recall	Per-class breakdown to catch underperforming categories	Per class	Secondary
Confusion matrix	Visual error analysis per toxicity category	Visual	Diagnostic

4. System Architecture and Deployment Plan

4.1 High-Level Design

The system runs entirely on free-tier infrastructure. Training takes place on Google Colab using a free T4 GPU session; the fine-tuned model weights are saved locally and uploaded to Amazon S3 when training is complete. The inference layer runs on a single AWS EC2 t2.micro instance: a Docker container hosts the FastAPI backend, which loads the DistilBERT model (Sanh et al., 2019) and serves predictions through a REST endpoint. A React frontend on the same instance provides a clean interface for live demonstration.

The t2.micro instance has 1 GB of RAM. DistilBERT occupies approximately 250 MB on disk, leaving enough headroom for the OS and application runtime. As a precaution, a 4 GB Linux swap partition is configured on the EBS volume—the same approach used in production low-memory deployments to prevent out-of-memory faults on cold inference requests.

Figure 2*System Architecture: API Inference Flow on AWS EC2*

Note. The client submits a POST request to the FastAPI backend; the model returns per-class confidence scores as a JSON payload. DistilBERT weights are loaded from Amazon S3 on container startup and held in memory for subsequent requests.

4.2 Infrastructure

Table 6 maps each infrastructure component to its technology and justification.

Table 6*Infrastructure Components, Technologies, and Justification*

Component	Technology	Justification
Model training	Google Colab (free T4 GPU)	Free GPU access; no cloud training costs; model exported after training
Dataset & model storage	Amazon S3 (free tier 5 GB)	Model weights ~250 MB; fits within the free-tier limit
Hosting	AWS EC2 t2.micro (free tier for 12 months)	1 GB RAM is sufficient for DistilBERT with swap; always-on for demos
Memory management	4 GB Linux swap on EBS	Prevents out-of-memory faults during inference on a 1 GB RAM instance

Component	Technology	Justification
Containerization	Docker	Reproducible environment; eliminates dependency drift
Backend framework	FastAPI	Async request handling; auto-generated OpenAPI docs; lightweight
Frontend	React (served from EC2)	Clean UI for text submission and per-category result display
Instance storage	Amazon EBS 20 GB (free tier)	OS + model weights + swap partition
Version control & CI/CD	GitHub + GitHub Actions (free)	Automated testing on every push; free for public repositories

4.3 API Contract

The inference endpoint accepts a POST request to /analyze with a JSON body containing a text field. It returns a JSON object with a scores field six key-value pairs mapping each category name to a float between 0.0 and 1.0 and a flagged field that applies a per-category threshold to produce binary decisions. A /batch endpoint accepts an array of strings and returns the same structure for each item.

4.4 Trade-off Analysis

Table 7 documents the key architectural decisions, the alternatives considered, and the reasoning behind each choice.

Table 7

Architectural Trade-off Decisions and Rationale

Decision	Alternative Considered	Why We Chose This
DistilBERT	RoBERTa-base	RoBERTa (~500 MB) strains t2.micro RAM; DistilBERT fits within free-tier constraints while retaining 97% of BERT accuracy (Sanh et al., 2019)

Decision	Alternative Considered	Why We Chose This
Google Colab for training	AWS SageMaker	SageMaker GPU instances are not free; Colab provides equivalent T4 GPU access at no cost
EC2 t2.micro (always-on)	AWS Lambda (serverless)	Lambda has a 250 MB deployment package limit, requiring cold-start model loading from S3; EC2 is simpler and free for 12 months
FastAPI	Flask	FastAPI handles async requests natively and auto-generates OpenAPI documentation
React frontend	Streamlit	React provides a cleaner, more professional interface; Streamlit is faster to build but less appropriate for a polished demo
EC2 t2.micro hosting	Hugging Face Spaces	HF Spaces restricts Docker configuration, does not expose a clean custom REST endpoint, and sleeps on inactivity; EC2 provides full control, a persistent always-on API, and is free for 12 months

5. Team Workflow and Project Management

Version Control

- Platform: GitHub (private repository)
- Branching: feature-branch workflow so no direct commits to main
- Merge policy: pull requests require one peer review before merge

Task Management

- Tool: GitHub Projects (Kanban board)
- Columns: To Do → In Progress → Review / QA → Done
- Cadence: weekly sprint planning at the start of each lab session

Communication

- Synchronous: weekly in-person meetings during scheduled lab sessions
- Asynchronous: WhatsApp group for daily updates, task check-ins, and quick decisions between meetings

Responsibilities

Table 8 maps each team member to their primary project responsibilities.

Table 8

Team Member Responsibilities

Team Member	Primary Responsibilities
Jerin Pious	Model architecture, fine-tuning on Colab, evaluation, SHAP explainability, experiment tracking
Evan Tomy	Data preprocessing pipeline, EC2 provisioning, Docker containerization, FastAPI backend, React frontend

6. Weekly Development Timeline

Table 9 presents the eight-week development schedule from Phase 1 submission through final project delivery.

Table 9

Weekly Development Milestones and Owners

Week	Dates	Milestone	Owner
7	Jun 23–29	Phase 1 proposal submitted. Presentation delivered. GitHub repo initialized. Dataset downloaded and explored.	All
8	Jun 30–Jul 6	Data preprocessing pipeline complete. Tokenizer integrated. Train/val/test splits finalized.	Jerin / Evan
9	Jul 7–13	Baseline models trained (TF-IDF + LR, LSTM). Initial benchmark F1 and ROC-AUC recorded.	Jerin
10	Jul 14–20	DistilBERT fine-tuning complete on Google Colab. Hyperparameter sweep. Model weights exported to S3.	Jerin

Week	Dates	Milestone	Owner
11	Jul 21–27	SHAP explainability analysis. Sigmoid threshold optimization per class. Final model locked.	Jerin
12	Jul 28–Aug 3	EC2 t2.micro provisioned. Docker container built and deployed. FastAPI endpoints verified end-to-end.	Evan Tomy
13	Aug 4–10	React frontend integrated. End-to-end user flow tested. Performance benchmarks documented.	All
14	Aug 11–17	Final demo delivered. Full project report submitted.	All

7. Technology Stack Summary

Table 10 provides a complete inventory of the technology stack used across all layers of the system.

Table 10

Full Technology Stack by Layer

Layer	Technology
Language	Python 3.12
ML Framework	PyTorch + Hugging Face Transformers
Base Model	distilbert-base-uncased (Hugging Face)
Data Processing	Pandas, NumPy, scikit-learn
Explainability	SHAP
Training Environment	Google Colab (free T4 GPU)
Backend API	FastAPI
Frontend	React
Containerization	Docker
Cloud Hosting	AWS EC2 t2.micro (free tier)
Model Storage	Amazon S3 (free tier — 5 GB)

Layer	Technology
Instance Storage	Amazon EBS 20 GB (free tier) + 4 GB swap partition
Version Control	GitHub
Task Tracking	GitHub Projects (Kanban)
Communication	WhatsApp + in-person lab sessions

References

- European Parliament. (2022). *Regulation (EU) 2022/2065 of the European Parliament and of the Council on a Single Market for Digital Services (Digital Services Act)*. Official Journal of the European Union. <https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:32022R2065>
- Government of Canada. (2024). *An Act to enact the Online Harms Act (Bill C-63)*. House of Commons. <https://www.canada.ca/en/canadian-heritage/services/online-harms.html>
- Jigsaw/Conversation AI. (2018). *Toxic comment classification challenge*. Kaggle. <https://www.kaggle.com/c/jigsaw-toxic-comment-classification-challenge>
- Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. *Proceedings of the 31st International Conference on Neural Information Processing Systems*, 4768–4777.
- Pennington, J., Socher, R., & Manning, C. D. (2014). GloVe: Global vectors for word representation. *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*, 1532–1543. <https://nlp.stanford.edu/projects/glove/>
- Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). *DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter*. arXiv preprint arXiv:1910.01108. <https://arxiv.org/abs/1910.01108>